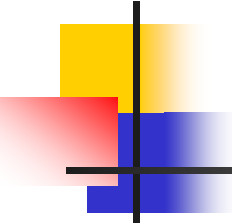


# 算法设计与分析

Analysis and Design of Algorithm

## Lesson 09



# 要点回顾

---

- 动态规划算法的设计要素
  - 建模：目标函数、约束条件
  - 分段：确定子问题的边界
  - 分析：子问题之间的依赖关系
  - 判断：最优子结构性质
  - 求解：先定最小子问题（初值），自底向上求解
- 实例：
  - 矩阵链相乘问题（迭代实现）
  - 组合问题中的动态规划法
    - 最长公共子序列
    - 背包问题

# LCS的递推方程/状态转移方程

- 令X和Y的子序列
- $X_i = \{x_1, x_2, \dots, x_i\}$  ,  $Y_j = \{y_1, y_2, \dots, y_j\}$
- $C[i, j]$ :  $X_i$ 与 $Y_j$ 的LCS的长度

$$C[i, j] = \begin{cases} 0 & \text{若 } i=0 \text{ 或 } j=0 \\ C[i-1, j-1]+1 & \text{若 } i, j > 0, x_i = y_j \\ \max \{C[i, j-1], C[i-1, j]\} & \text{若 } i, j > 0, x_i \neq y_j \end{cases}$$

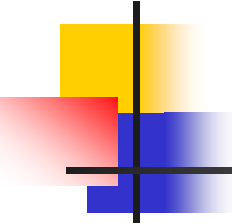
# KP的递推方程/状态转移方程

$F_k(y)$ : 装前  $k$  种物品, 总重不超过  $y$   
背包达到的最大价值

$$F_k(y) = \max\{F_{k-1}(y), F_k(y-w_k)+v_k\}$$

$$F_0(y) = 0, 0 \leq y \leq b, F_k(0) = 0, 0 \leq k \leq n$$

$$F_1(y) = \left\lfloor \frac{y}{w_1} \right\rfloor v_1, \quad F_k(y) = -\infty \quad y < 0$$



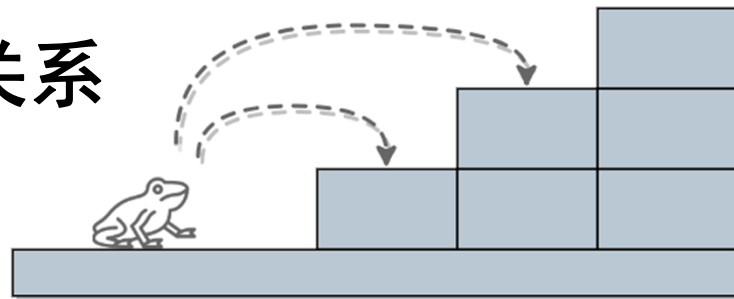
# 背包问题的推广

- **物品数受限**背包：第 $i$ 种物品最多用 $n_i$ 个
- **0-1**背包问题：  $x_i=0,1, i=1, 2, \dots, n$
- **多背包**问题：  $m$ 个背包，背包 $j$ 装入最大重量 $B_j, j=1, 2, \dots, m$ 。在满足所有背包重量约束条件下使装入物品价值最大。
- **二维背包**问题： 每件物品有重量 $w_i$ 和体积 $t_i$ ， $i=1, 2, \dots, n$ ，背包总重不超过 $b$ ，体积不超过 $V$ ，如何选择物品以得到最大价值。

# 走台阶问题

- 假设在一只青蛙面前有 $n$ 阶台阶。对于青蛙而言，每步要么往上跳1阶，要么往上跳2阶。请用动态规划求解：跳到第 $n$ 阶时，共有多少种跳法？

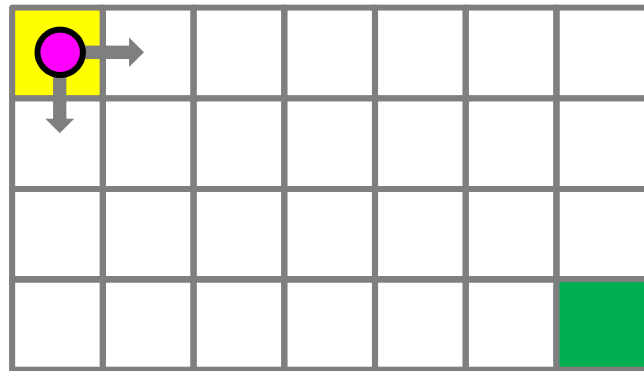
- 第一步：子问题界定
- 第二步：子问题间的依赖关系
- 第三步：自底向上求解



动态规划法求解斐波那契数！

# 走方格问题

- 给定一个矩形，有 $n*m$ 个方格组成。起点为最左上角的(1,1)，假设一扫地机器人每步要么只能往下走1格，要么只能往右走1格。请用动态规划求解，当机器人走到最右下角( $n,m$ )时，共有多少种走法？



# 走方格问题

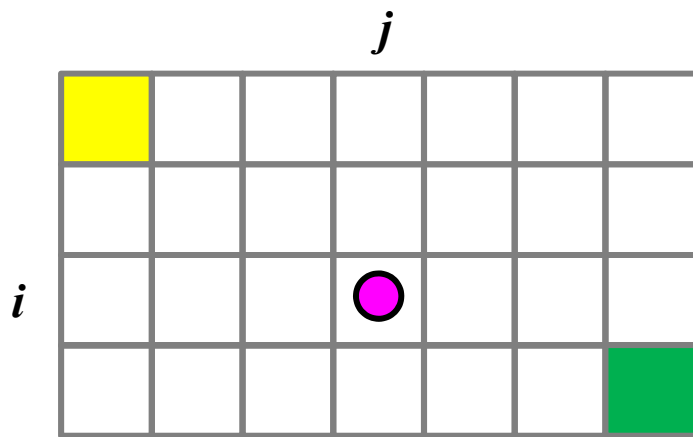
- 给定一个矩形，有 $n*m$ 个方格组成。起点为最左上角的(1,1)，假设一扫地机器人每步要么只能往下走1格，要么只能往右走1格。请用动态规划求解，当机器人走到最右下角( $n,m$ )时，共有多少种走法？

- **第一步：子问题界定**

用参数  $i$  和  $j$  界定子问题



$f(i, j)$ 表示从起点走到第( $i, j$ )方格的总走法

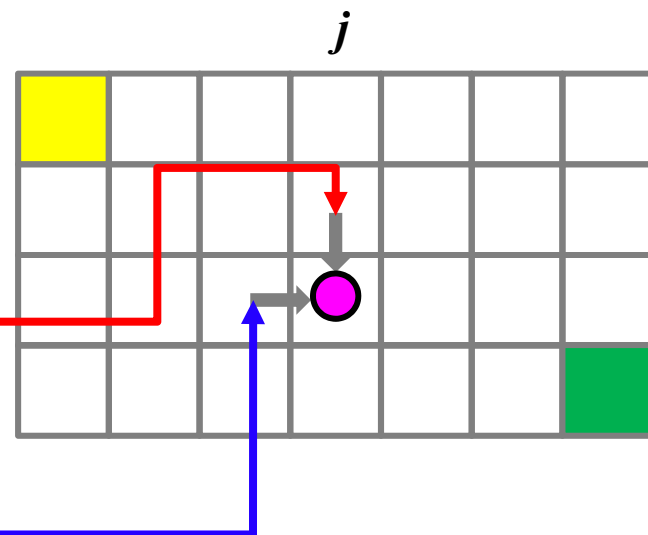


# 走方格问题

- 给定一个矩形，有 $n*m$ 个方格组成。起点为最左上角的 $(1,1)$ ，假设一扫地机器人每步要么只能往下走1格，要么只能往右走1格。请用动态规划求解，当机器人走到最右下角 $(n,m)$ 时，共有多少种走法？

- 第二步：子问题间的依赖关系

$$f(i, j) = f(i-1, j) + f(i, j-1)$$



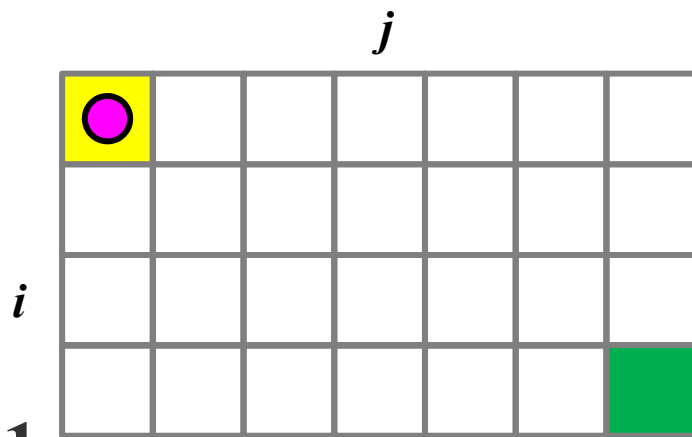
# 走方格问题

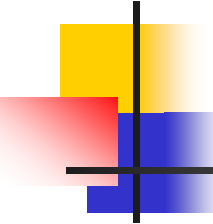
- 给定一个矩形，有 $n*m$ 个方格组成。起点为最左上角的 $(1,1)$ ，假设一扫地机器人每步要么只能往下走1格，要么只能往右走1格。请用动态规划求解，当机器人走到最右下角 $(n,m)$ 时，共有多少种走法？

- **第三步：**自底向上求解

$$f(1, 1) = 1$$

$$f(0, j) = 0, f(i, 0) = 0, \forall i, j \geq 1$$





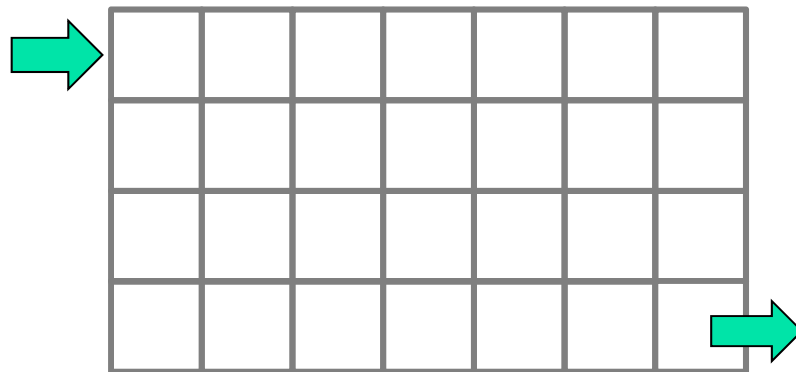
# 算法代码

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main()
4 {
5     //动态规划求解走方格问题
6     f[1][1]=1;
7     int n,m;
8     cin>>n>>m;
9     for (int i=1;i<=n;i++)
10         for (int j=1;j<=m;j++)
11             if (i!=1 || j!=1) f[i][j]=f[i-1][j]+f[i][j-1];
12     cout<<f[n][m];
13     return 0;
14 }
```

# 拓展思考1

## ■ 寻宝问题

给定一个 $n*m$ 方格矩形，每个方格 $(i, j)$ 上堆了一份价值为 $v(i, j)$ 的宝物。假设寻宝人从起点 $(1, 1)$ 出发，每次只可以向下或向右移动，如何行走才能使得获得宝物的价值最大，并顺利从出口 $(n, m)$ 离开？



# 拓展思考2

## ■ 机器人导航问题

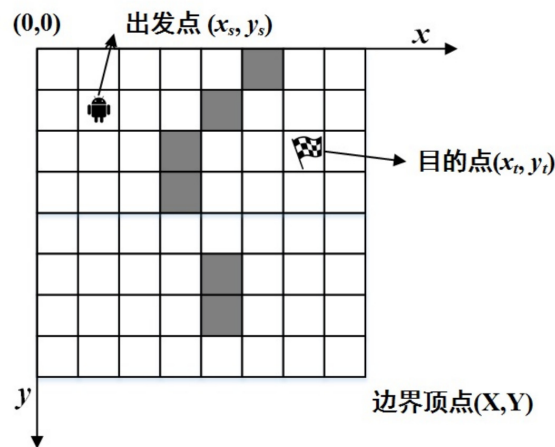


图 2 是一个二维矩形网格世界，其中每个格子可以用坐标  $(x, y)$  表示，边界顶点用  $(X, Y)$  表示。一个机器人需要从出发点  $(x_s, y_s)$  移动到目的点  $(x_t, y_t)$ 。机器人每一步只能上、下、左、右移动一格，其中灰色方格为障碍物，表示机器人无法通过。如果某一步的动作使得机器人移出网格世界，则机器人保持上一步所在的方格位置。机器人希望以最少的步长移动到目的点。定义  $Q(x, y)$  为机器人所在方格为  $(x, y)$  时，到目的点  $(x_t, y_t)$  的最短步长，其中当  $(x, y)$  为灰色方格时， $Q(x, y) = \infty$ ，问：

- (1) 在图 2 中， $Q(x_t + 1, y_t) = ?$   $Q(x_t, y_t - 1) = ?$
- (2) 以  $Q(x_t, y_t) = 0$  为初始条件，试写出  $Q(x, y)$  的动态归纳公式。
- (3) 根据 (2) 中的动态归纳公式，试设计动态规划算法计算  $Q(x_s, y_s)$ 。

# 拓展思考3

## ■ 三维沙盒游戏问题

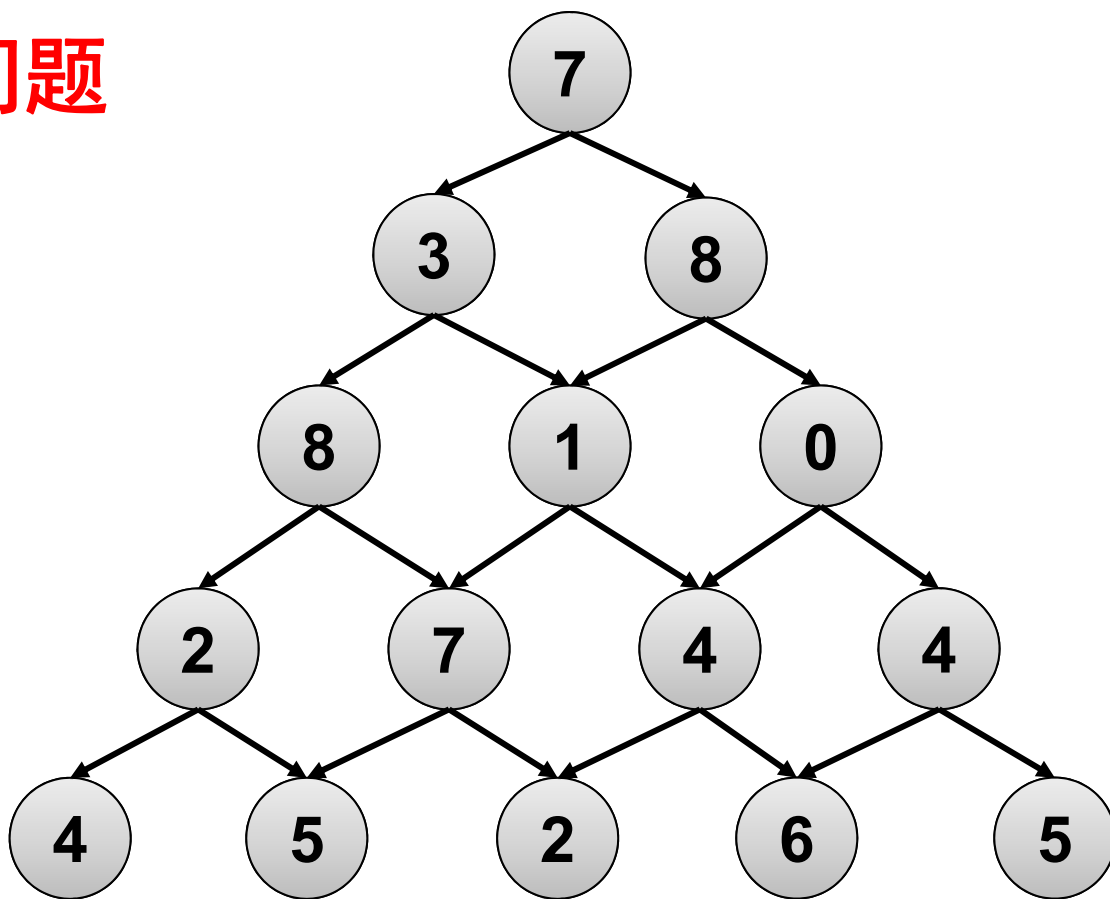


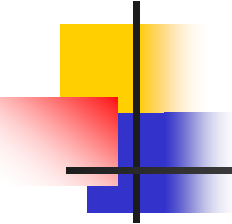
<https://www.luogu.com.cn/problem/CF1353F>

# 拓展思考4

## ■ 数字三角形问题

图中给出了一个数字三角形。从三角形的顶部到底部有很多条不同的路径。对于每条路径，把路径上面的数加起来可以得到一个和，和最大的路径称为最佳路径。你的任务就是求出最佳路径上的数字之和。

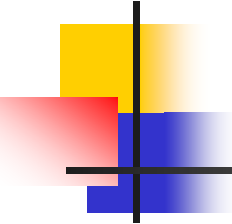




## 第三章小结

---

- 理解动态规划算法的概念
- 掌握动态规划算法的基本要素
  - 最优子结构性质
  - 重叠子问题性质
- 掌握设计动态规划算法的步骤
  - 建模 → 分段 → 分析 → 判断 → 求解
- 动态规划法应用实例
  - 多源最短路径、矩阵连乘
  - LCS问题、背包问题、Grid问题



# 课程内容

NP完全性理论与近似算法

算法高级理论

随机化算法

线性规划与网络流

高级算法

递归  
分治

动态  
规划

贪心  
算法

回溯与  
分支限界

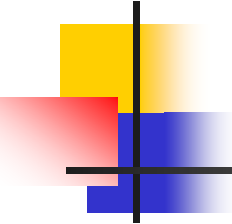
基础算法

算法分析与问题的计算复杂性

算法基础理论

# 第四章 贪心算法

Greedy Algorithm

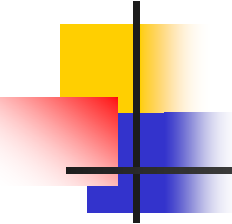


# 学习要点

---

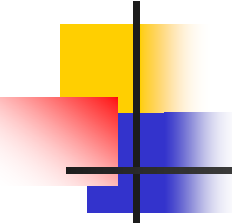
- 理解贪心算法的概念，掌握贪心算法的基本要素
  - 最优子结构性质
  - 贪心选择性质
- 理解贪心算法一般理论，与动态规划算法的差异
- 通过应用范例学习贪心设计策略
  1. 一般背包问题
  2. 活动安排/选择问题
  3. 最优装载问题
  4. 哈夫曼编码
  5. 单源最短路径
  6. 最小生成树

# 贪心算法概述



# 贪心算法(Greedy)

- 顾名思义，贪心算法总是作出在当前阶段看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择，是一种“**短视**”行为。
  - 希望贪心算法得到的最终结果也是整体最优的。
- 不能对所有问题都得到整体最优解，但对许多问题能产生整体最优解。
  - 如单源最短路径问题，最小生成树问题等。
  - 在一些情况下，即使不能得到整体最优解，其最终结果却是最优解的很好**近似**。



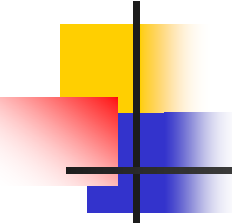
# 贪心算法思想

- 从一个实例开始：找零钱问题

设有多张5、2、1元和5、2、1角货币，需找给顾客4元6角现金，如何支付能使付出的货币数目最少？

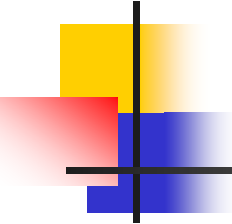
1. 首先选出1张最大面值不超过4元6角→2元；
2. 再选2元6角→2元；
3. 再选不超过6角的最大面值→5角；
4. 再选一张不超过1角的→1角；

共付出4张货币。



# 找零钱问题的策略

- 在每一步贪心选择中，在不超过应找零钱的条件下，只选择面值最大的货币，不考虑选择的合理性，且不会改变决定：一旦选出了一张货币，就永远选定。
  - 策略：尽可能使付出的货币最快地满足支付要求，目的是使付出的货币张数最慢增加。
  - 上述实例用贪心法可以得到最优解。
  - 但是，将面值改为3元、1元、8角、5角、1角
  - **贪心法**：结果是3元、1元、5角和1角各一张，共**4**张。
  - **最优解**：却为**1**个3元+**2**个8角，共**3**张。



# 贪心法求解的问题的特征

- 最优子结构性质

- 当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**，也称此问题满足最优性原理/优化原则。

- 贪心选择性质

- 所谓贪心选择性质是指问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来得到。

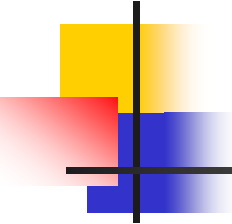
动态规划实质：

- 以**自底向上**的方式求解各个子问题

贪心算法实质：

- 以**自顶向下**的方式做出一系列的贪心选择

# 组合问题中的贪心算法



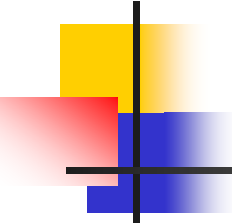
# 一般背包问题

- 一般背包问题的解是 $\langle x_1, x_2, \dots, x_n \rangle$ ，其中 $x_i$ 是装入背包的第 $i$ 种物品一部分，不一定要全部装入背包

**目标函数**  $\max \sum_{i=1}^n v_i x_i$

**约束条件**  $\sum_{i=1}^n w_i x_i \leq b, \quad x_i \in [0, 1]$

- 当 $x_i \in \{0, 1\}$ 时，变为0-1背包问题的解
- 背包问题具有最优子结构性质！



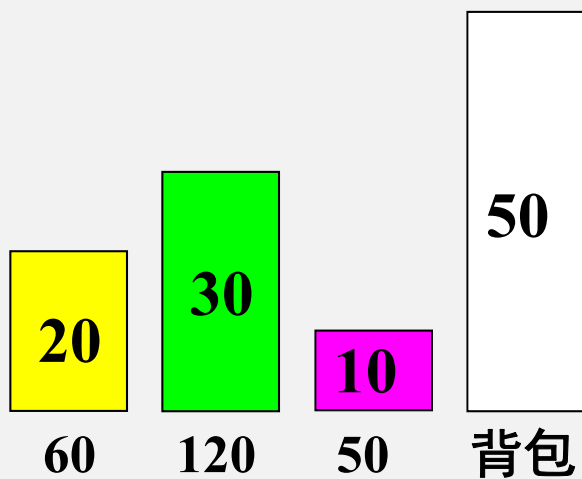
# 背包问题的贪心法

- 至少有三种看似合理的贪心策略：
  1. 选择**价值最大**的物品，因为这可以尽可能快地增加背包的总价值。然而，虽然每一步选择获得了背包价值的极大增长，但背包容量却可能消耗得太快，使得装入背包的物品个数减少，从而不能保证目标函数达到最大。
  2. 选择**重量最轻**的物品，因为这可以装入尽可能多的物品，从而增加背包的总价值。但是，虽然每一步选择使背包的容量消耗得慢了，但背包的价值却没能保证迅速增长，从而不能保证目标函数达到最大。
  3. 选择**单位重量价值最大**的物品，在背包价值增长和背包容量消耗两者之间寻找平衡。

# 背包问题的贪心法

## ■ 一般背包问题实例：

- 有3个物品，其重量分别是{20, 30, 10}，价值分别为{60, 120, 50}，背包的容量为50。应用三种贪心策略装入背包的物品和获得的价值如图所示。

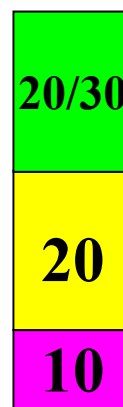


三个物品及背包



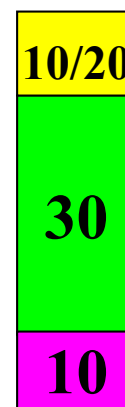
180

贪心策略1



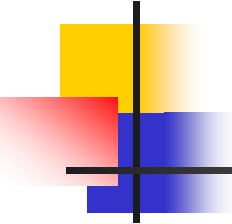
190

贪心策略2



200

贪心策略3



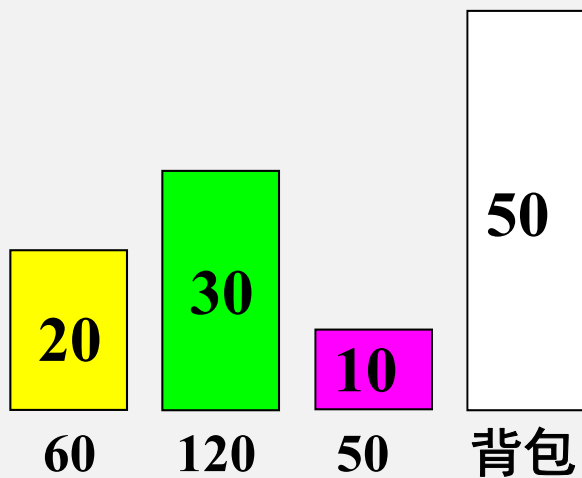
# 背包问题的贪心法

- 应用第**3**种贪心策略，每次从物品集合中选择**性价比最高**的物品，如果其重量小于背包容量，就可以把它装入，并将背包容量减去该物品的重量。
- 然后就面临一个最优子问题——同样是背包问题，只不过背包容量减少了，物品集合减少了。
- 因此背包问题具有最优子结构性质。

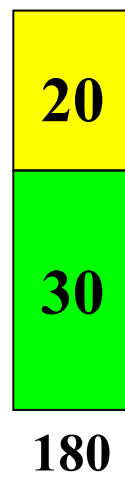
# 背包问题的贪心法

## ■ 0-1背包问题实例：

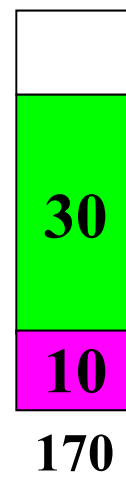
- 有3个物品，其重量分别是{20, 30, 10}，价值分别为{60, 120, 50}，背包的容量为50。采用第3种贪心策略装入背包的物品和获得的价值如图所示。



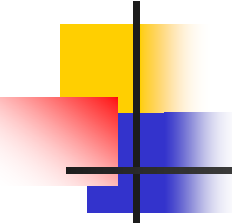
三个物品及背包



最优解



贪心策略3



# 背包问题的贪心法

## ■ 结论：

- 对于0-1背包问题，贪心选择之所以不能得到最优解，是因为在这种情况下，无法保证最终能将背包装满，部分闲置的背包空间使每千克背包空间的价值降低了。
- 事实上，在考虑0-1背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再做出最好选择。
- 由此可导出许多互相重叠的子问题。这正是该问题可以用动态规划算法求解的另一重要特征。动态规划算法可以有效解决0-1背包问题。

# 活动选择问题



# 活动选择问题

**问题实例：** 有11个活动，每个活动的时间表如下

极坐标话剧团 (10:00-22:00)

唐仲英爱心社 (16:00-21:00)

机器人俱乐部 (16:00-20:00)

创行 (14:00-18:00)

电子竞技协会 (20:00-23:00)

魔方协会 (11:00-17:00)

东南大学电视台 (08:00-15:00)

辩论协会 (09:00-13:00)

大学生心理健康协会 (11:00-14:00)

跆拳道协会 (14:00-19:00)

华风汉韵文化社 (13:00-16:00)



# 活动选择问题

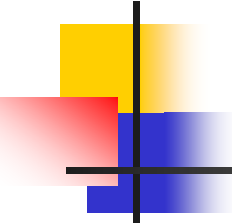
- **输入：**  $S=\{1,2,\dots,n\}$  为  $n$  项活动的集合， $s_i, f_i$  分别为活动  $i$  的开始和结束时间。

活动  $i$  与  $j$  相容  $\leftrightarrow s_i \geq f_j$  或  $s_j \geq f_i$

- **输出：** 最大的两两相容的活动集  $A$
- **实例：**

| $i$   | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 |
|-------|---|---|---|---|---|---|----|----|----|----|
| $s_i$ | 1 | 3 | 2 | 5 | 4 | 5 | 6  | 8  | 8  | 2  |
| $f_i$ | 4 | 5 | 6 | 7 | 9 | 9 | 10 | 11 | 12 | 13 |

**解：**  $A=\{1,4,8\}$



# 贪心算法

挑选过程是多步判断过程，每步依据某种“短视”的策略进行活动的选择，选择时候需要满足相容性条件。

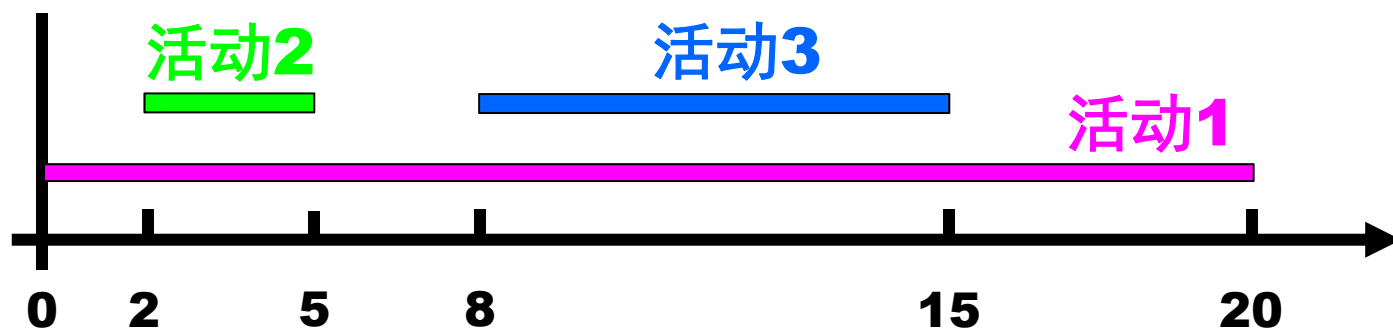
- 策略1：开始时间早的优先
  - 排序使  $s_1 \leq s_2 \leq \dots \leq s_n$ ，从前往后挑选
- 策略2：占用时间少的优先
  - 排序使  $f_1 - s_1 \leq f_2 - s_2 \leq \dots \leq f_n - s_n$ ，从前往后挑选
- 策略3：结束时间早的优先
  - 排序使  $f_1 \leq f_2 \leq \dots \leq f_n$ ，从前往后挑选

# 策略1的反例

- 策略1：开始早的优先

- 反例：  $S=\{1,2,3\}$

$$s_1=0, f_1=20, s_2=2, f_2=5, s_3=8, f_3=15$$

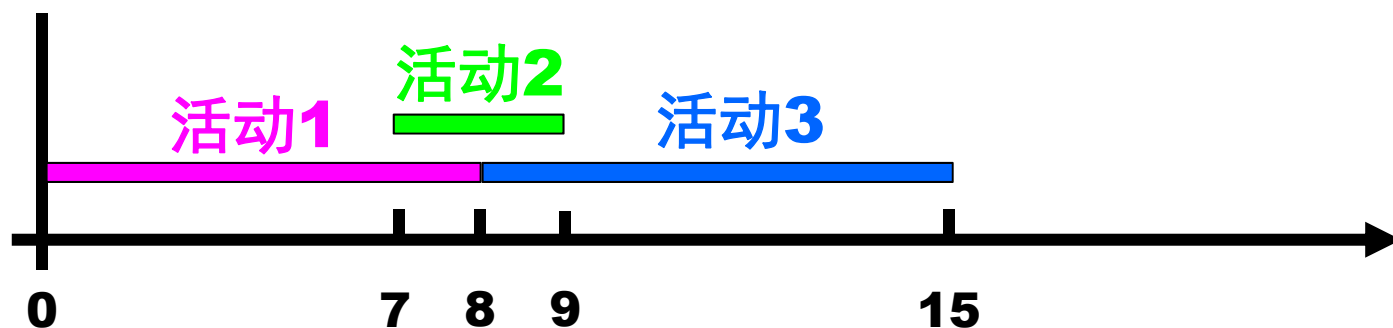


# 策略2的反例

- 策略2：占时少的优先

- 反例：  $S=\{1,2,3\}$

$$s_1=0, f_1=8, s_2=7, f_2=9, s_3=8, f_3=15$$



# 策略3伪码

- 算法GreedySelect
  - 输入：活动集S,  $s_i, f_i, i=1,2,\dots,n, f_1 \leq f_2 \leq \dots \leq f_n$
  - 输出：  $A \subseteq S$ ，选中的活动子集
1.  $n \leftarrow \text{length}[S]$
  2.  $A \leftarrow \{1\}$
  3.  $j \leftarrow 1$
  4. **for**  $i \leftarrow 2$  **to**  $n$  **do**
  5.     **if**  $s_i \geq f_j$
  6.     **then**  $A \leftarrow A \cup \{i\}$
  7.      $j \leftarrow i$
  8. **return**  $A$

已选入的  
最后标号

判断  
相容性

完成时间  $t = \max \{f_k : k \in A\}$

# 运行实例

- 输入：  $S=\{1,2, \dots, 10\}$

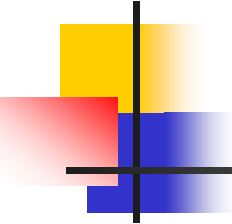
| $i$   | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 |
|-------|---|---|---|---|---|---|----|----|----|----|
| $s_i$ | 1 | 3 | 0 | 5 | 3 | 5 | 6  | 8  | 8  | 2  |
| $f_i$ | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |

解：  $A=\{1,4,8\}$ ，完成时间  $t = 11$

时间复杂度：

$$O(n\log n)+O(n)=O(n\log n)$$

问题：如何证明该算法对所有实例都得到正确的解？



# 贪心算法的特点

- 设计要素：
  - 贪心法适用于组合优化问题
  - 求解过程是多步判断过程，最终的判断序列对应于问题的最优解
  - 依据某种“短视”的贪心选择性质判断，性质好坏决定算法的成败
  - 贪心法必须进行正确性证明
  - 证明贪心法不正确的技巧：反证法

贪心优势：算法简单，时空复杂度低

# 贪心法正确性证明

## ■ 数学归纳法

■ **例：** 证明对于任何自然数 $n$ ,

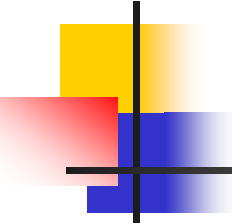
$$1+2+\dots+n=n(n+1)/2$$

■ **证：**  $n=1$ , 左边=1, 右边= $1 \times (1+1)/2=1$   
假设对于任意自然数 $n$ 等式成立, 则

$$\begin{aligned} &1+2+\dots+(n+1) \\ &= (1+2+\dots+n)+(n+1) \\ &= n(n+1)/2+(n+1) \\ &= (n+1)(n/2+1) \\ &= (n+1)(n+2)/2 \\ &= \color{red}{(n+1)}(\color{red}{(n+1)}+1)/2 \end{aligned}$$

代入  
归纳假设





# 第一数学归纳法

适合证明涉及自然数的命题  $P(n)$

- **归纳基础**: 证明  $P(1)$  为真 (或  $P(0)$  为真)
- **归纳步骤**: 若对所有  $n$  有  $P(n)$  为真, 证明  $P(n+1)$  为真

$$\forall n, P(n) \rightarrow P(n+1)$$

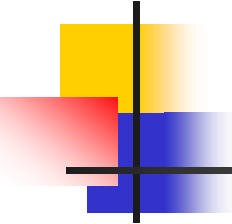
$$P(1)$$

$$n=1, P(1) \Rightarrow P(2)$$

$$n=2, P(2) \Rightarrow P(3)$$

$$n=3, P(3) \Rightarrow P(4)$$

.....



# 第二数学归纳法

适合证明涉及自然数的命题  $P(n)$

- **归纳基础**: 证明  $P(1)$  为真 (或  $P(0)$  为真)
- **归纳步骤**: 若对所有小于  $n$  的  $k$  有  $P(k)$  为真, 证明  $P(n)$  为真

$$\forall k (k < n, P(k)) \rightarrow P(n)$$

$$P(1)$$

$$n=2, P(1) \Rightarrow P(2)$$

$$n=3, P(1) \wedge P(2) \Rightarrow P(3)$$

$$n=4, P(1) \wedge P(2) \wedge P(3) \Rightarrow P(4)$$

.....

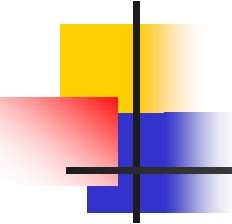
# 两种归纳法的区别

- 归纳基础一样      都是 $P(1/0)$ 为真
- 归纳步骤不同

## 证明逻辑

- 归纳法1:  $P(1) \Rightarrow P(2) \Rightarrow P(3) \Rightarrow \dots$
- 归纳法2:

$$\begin{array}{c} P(1) \\ P(1) \Rightarrow P(2) \end{array} \Bigg\} \Rightarrow P(3) \quad \begin{array}{c} P(1) \\ P(2) \end{array} \Bigg\} \Rightarrow P(4) \Rightarrow \dots$$



# 贪心算法正确性归纳证明

## ■ 证明步骤

1. 叙述一个有关自然数 $n$ 的**命题**，该命题断定该贪心策略的执行最终将导致最优解。其中自然数 $n$ 可以代表步数或问题规模
2. 证明命题对所有的自然数为真
  - **2.1 归纳基础**（从最小实例规模开始）
  - **2.2 归纳步骤**（第一或第二数学归纳法）

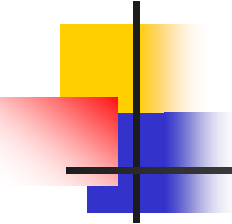
# 活动选择贪心策略3伪码

- 算法GreedySelect
- 输入：活动集S,  $s_i, f_i, i=1,2,\dots,n, f_1 \leq f_2 \leq \dots \leq f_n$
- 输出：  $A \subseteq S$ ，选中的活动子集

```
1.  $n \leftarrow \text{length}[S]$ 
2.  $A \leftarrow \{1\}$ 
3.  $j \leftarrow 1$ 
4. for  $i \leftarrow 2$  to  $n$  do
5.     if  $s_i \geq f_j$ 
6.     then  $A \leftarrow A \cup \{i\}$ 
7.          $j \leftarrow i$ 
8. return A
```

如何证明策略3的正确性？

完成时间  $t = \max \{f_k : k \in A\}$



# 活动选择算法的命题

## 1. 命题：

算法GreedySelect执行到第  $k$  步，选择  $k$  项活动

$i_1, i_2, \dots, i_k$

则存在最优解 A 包含活动  $i_1=1, i_2, \dots, i_k$ 。

根据上述命题：对于任何  $k$ ，算法前  $k$  步的选择都将导致最优解，至多到第  $n$  步将得到问题实例的最优解。

# 归纳法证明

令  $S=\{1,2,\dots,n\}$  是活动集，且  $f_1 \leq f_2 \leq \dots \leq f_n$

- **2.1 归纳基础：**  $k=1$ , 证明存在最优解包含 **活动1**

- **证明：**

- 任取最优解  $A$ ， $A$  中活动按结束时间递增排序。如果  $A$  的第一个活动为  $j$ ，且  $j \neq 1$ ，用 1 替换  $A$  的活动  $j$  得到解  $A'$ ，即  $A'=(A-\{j\}) \cup \{1\}$
- 由于  $f_1 \leq f_j$ ， $A'$  也是最优解，且含有 1。

